

Rapport du Projet de Grammaire et Langage

Travaux Pratiques : Algoritme de transformation AFN vers AFD et Algorithme de minimisation

Réalisé par :

Djibrine Moustapha Mahamat

Khadidja Ahmat Hassan

Sous la direction de :

Mr. Philippe Gaborit

Année académique : 2024-2025

I. Présentation du travail

Ce rapport présente le travail réalisé pour le TP de grammaires et langages, centré sur la conception de deux algorithmes en Java. Le premier algorithme permettra de transformer un automate fini non déterministe (AFN) en un automate fini déterministe (AFD), tandis que le second se concentrera sur la minimisation d'un automate fini déterministe (AFD) donné en réduisant le nombre d'états.

L'objectif de ce projet est de spécifier clairement les entrées et les sorties des algorithmes. Pour le premier algorithme, l'entrée sera un fichier texte contenant la description de l'automate fini non déterministe (AFN), incluant ses états, symboles et transitions. La sortie sera un automate fini déterministe (AFD) correspondant. Pour le second algorithme, l'entrée sera un AFD, et la sortie sera l'AFD minimisé, tout en conservant le même langage accepté.

II. Description du principe de l'algorithme et du choix des structures utilisées

❖ Principe de l'algorithme

L'algorithme utilisé implémente la conversion d'un Automate Finis Non Déterministe (AFN) en un Automate Finis Déterministe (AFD). Le principe repose sur l'algorithme de construction des sous-ensembles (ou méthode de l'algorithme de subset construction). Cette méthode consiste à créer un AFD dont chaque état correspond à un ensemble d'états de l'AFN, permettant ainsi de rendre l'automate déterministe.

Les étapes principales de l'algorithme sont :

- **Initialisation** : On commence par créer un état initial pour l'AFD qui contient uniquement l'état initial de l'AFN.
- **Analyse des états** : Pour chaque état de l'AFD, on examine les transitions possibles sur chaque symbole 'a' et 'b'. Cela donne un nouvel ensemble d'états dans l'AFD.
- **Définition des transitions** : Les transitions de l'AFD sont définies en fonction des transitions de l'AFN pour chaque ensemble d'états.
- **Marquage des états finaux** : Un état de l'AFD est marqué comme final si au moins un des états de l'ensemble correspondant est un état final dans l'AFN.

❖ Choix des structures utilisées

Pour mettre en œuvre l'algorithme de construction des sous-ensembles, les structures suivantes sont utilisées:

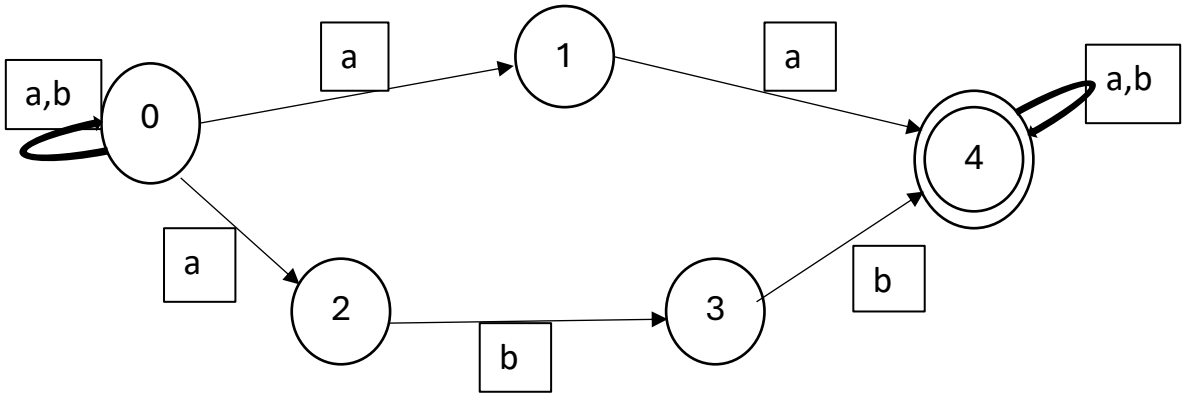
- **Table de transitions :** C'est une structure de données à trois dimensions qui est utilisée pour représenter les transitions de l'AFN. Les deux premières dimensions sont utilisées pour représenter les états et les symboles de l'alphabet ('a' et 'b'). La troisième dimension contient les états de destination pour chaque transition.
- **Listes de transitions (transitions)** : Une liste de listes est utilisée pour stocker les transitions de chaque état pour chaque symbole. Cela permet de gérer facilement les transitions multiples de l'AFN.
- **File d'attente (Queue)** : Une **queue** est utilisée pour gérer les ensembles d'états à explorer, ce qui permet de simuler un parcours en largeur lors de la construction de l'AFD.
- **Ensembles (Set)** : Les ensembles sont utilisés pour représenter les états de l'AFD, car un état de l'AFD est un ensemble d'états de l'AFN.

Cet algorithme permet de transformer efficacement un AFN sans epsilon transitions en un AFD équivalent.

III. Implémentation de l'algorithme de AFN en AFD

Dans cette implémentation, nous avons conçu un programme en Java permettant de convertir un automate fini non déterministe (AFN) en un automate fini déterministe (AFD). Le programme commence par lire les données d'un fichier texte contenant la description de l'AFN (nombre d'états, état initial, états finaux, et transitions). Une fois les données extraites, elles sont structurées à l'aide de collections Java, notamment des Map et des Set, afin de représenter les états et transitions de manière optimale. Chaque nouvel état de l'AFD est identifié comme une combinaison d'états de l'AFN, et les transitions sont construites en conséquence. Enfin, le programme affiche les états, transitions, état initial, et états finaux de l'AFD, permettant une vérification claire et lisible des résultats.

Voici notre automate AFN :



Description de notre automate :

- **Nombre d'états** : 5 → États : {0, 1, 2, 3, 4}.
- **État initial** : 0 → L'état initial est 0.
- **État final** : 4 → L'état final est 4.
- **Transitions** : les transitions décrivent le comportement de l'AFN pour chaque symbole lu.
  Voici notre fichier afn.txt contenant la description de notre AFN ou la représentation de l'AFN :

```
a 0 0, b 0 0 : Boucles sur l'état 0 pour a et b.

a 0 1, a 0 2 : Transitions de 0 vers 1 et 2 sur a.

a 1 4 : Transition de 1 vers 4 sur a.

b 2 3 : Transition de 2 vers 3 sur b.

b 3 4 : Transition de 3 vers 4 sur b.

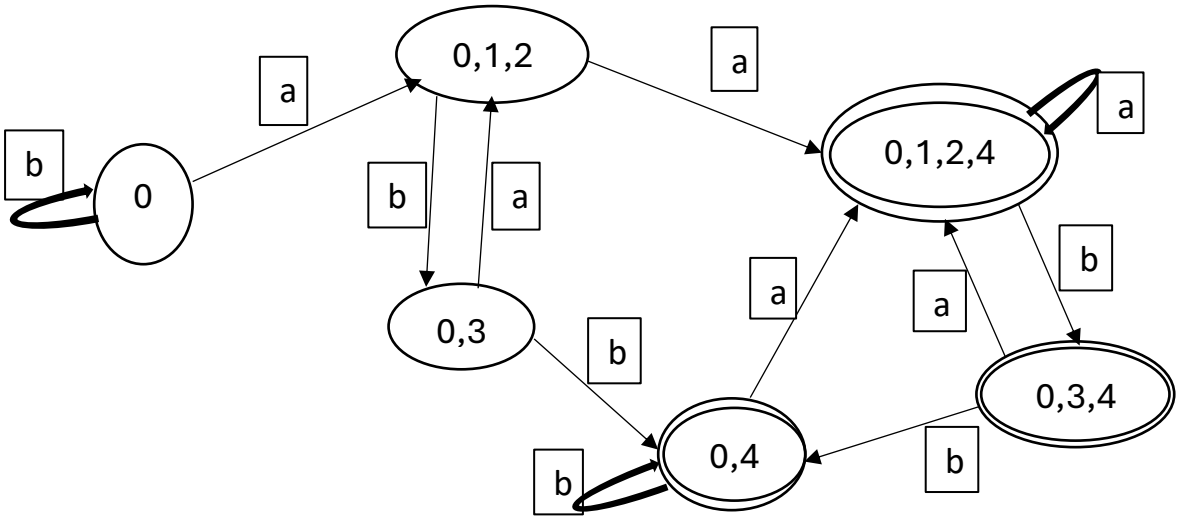
a 4 4, b 4 4 : Boucles sur l'état 4 pour a et b.
```

La table de transitions :

La table de transition est une représentation tabulaire des transitions d'un automate. Elle indique, pour chaque état et chaque symbole de l'alphabet, vers quel état l'automate se déplace lorsqu'il lit ce symbole.

État	a	b
[0]	**[0,1,2]**	**[0]**
[0,1,2]	**[0,1,2,4]**	**[0,3]**
[0,1,2,4]	**[0,1,2,4]**	**[0,3,4]**
[0,3]	**[0,1,2]**	**[0,4]**
[0,3,4]	**[0,1,2,4]**	**[0,4]**
[0,4]	**[0,1,2,4]**	**[0,4]**

Voici l'automate fini déterministe AFD :



Pour exécuter le code : javac AFNtoAFD.java, pour compiler java AFNtoAFD

Voici le résultat du code :

```
PS C:\Users\ahmat\OneDrive\Documents\Grammaire et langage>  & 'C:\Program Files\Java\jdk-23\bin\java.exe' '--enable-
wCodeDetailsInExceptionMessages' '-cp' 'C:\Users\ahmat\AppData\Roaming\Code\User\workspaceStorage\9ce32843039b88b008
hat.java\jdt_ws\Grammaire et langage_38b4fcfc\bin' 'AFNtoAFD'
Lecture du fichier afn.txt...
Nombre d'états : 5
État initial : 0
États finaux : [4]
Transitions de l'AFN : {0={a=[0, 1, 2], b=[0]}, 1={a=[], b=[]}, 2={a=[], b=[3]}, 3={a=[], b=[4]}, 4={a=[4], b=[4]}}
États de l'AFD : [[0], [0, 1, 2], [0, 3], [0, 4], [0, 1, 2, 4], [0, 3, 4]]
État initial de l'AFD : [0]
États finaux de l'AFD : [[0, 4], [0, 1, 2, 4], [0, 3, 4]]
Transitions de l'AFD :
Depuis [0] : a -> [0, 1, 2] b -> [0]
Depuis [0, 1, 2] : a -> [0, 1, 2] b -> [0, 3]
Depuis [0, 3] : a -> [0, 1, 2] b -> [0, 4]
Depuis [0, 4] : a -> [0, 1, 2, 4] b -> [0, 4]
Depuis [0, 1, 2, 4] : a -> [0, 1, 2, 4] b -> [0, 3, 4]
Depuis [0, 3, 4] : a -> [0, 1, 2, 4] b -> [0, 4]
PS C:\Users\ahmat\OneDrive\Documents\Grammaire et langage>
```

IV. Description du principe de l'algorithme de minimisation et du choix des structures utilisées

❖ Principe de l'algorithme de minimisation

L'algorithme de minimisation vise à réduire un automate fini déterministe (AFD) en un automate équivalent avec le nombre minimal d'états. Cette réduction est

réalisée en fusionnant les états équivalents, c'est-à-dire ceux qui ne peuvent pas être distingués par les mots de l'alphabet. Le principe repose sur l'identification et la séparation progressive des états en groupes distincts, appelés partitions.

Les étapes principales de l'algorithme sont :

- **Initialisation des partitions :** Les états de l'automate sont initialement divisés en deux groupes :
  - Le premier groupe contient les **états finaux**, qui acceptent les mots du langage.
  - Le second groupe contient les **états non finaux**, qui rejettent les mots.
- **Refinement des partitions :** Chaque groupe est analysé pour déterminer si des états doivent être séparés en sous-groupes. Pour cela, on vérifie les transitions de chaque état pour chaque symbole de l'alphabet. Si deux états d'un même groupe mènent vers des partitions différentes, ils sont placés dans des sous-groupes distincts. Ce processus est itératif et continue jusqu'à ce qu'aucune modification ne soit nécessaire.
- **Construction de l'automate minimal :** Une fois les partitions stabilisées, chaque groupe est traité comme un état unique dans le nouvel automate. Les transitions, l'état initial et les états finaux sont définis en fonction des partitions.

❖ **Choix des structures de données**

Pour implémenter efficacement l'algorithme de minimisation, il est crucial de sélectionner des structures de données adaptées à chaque étape. Voici les structures choisies et leur justification :

- **Représentation des transitions :** Les transitions de l'automate sont modélisées à l'aide d'une structure de type `Map<String, Map<Character, String>>`.
  - La première map associe chaque état à une seconde map.
  - La seconde map relie chaque symbole de l'alphabet (`a`, `b`) à l'état cible correspondant.

Justification : Cette organisation permet une gestion claire et rapide des transitions. On peut accéder directement à l'état cible d'un état source pour un symbole donné, sans nécessiter de recherches complexes.

- **Partitions des états :** Les partitions, qui regroupent les états équivalents, sont représentées par une liste de sets (`List<Set<String>>`).
  - Chaque set contient un groupe d'états qui, à un instant donné, sont considérés comme équivalents.
  - La liste contient l'ensemble des partitions, chaque partition étant un set distinct.

Justification : Cette structure permet de maintenir un suivi clair des groupes d'états et facilite les vérifications, comme l'appartenance d'un état à une partition. De plus, elle simplifie le processus de raffinage en regroupant les états équivalents.

- **États finaux :** Les états finaux sont stockés dans un `Set<String>`.

Justification : Cette structure offre une recherche rapide pour vérifier si un état est final. Elle est également adaptée pour des opérations comme l'union ou la différence, utiles pour séparer les états finaux des non-finaux lors de l'initialisation.

- **Mapping des états minimisés :** Une map (`Map<String, String>`) est utilisée pour associer chaque état de l'automate original à son équivalent dans l'automate minimal.

Justification : Cette map est essentielle pour simplifier la construction de l'automate minimal. Elle garantit que chaque état de l'automate d'origine est correctement représenté dans la version réduite.

Les structures choisies offrent un équilibre entre simplicité et efficacité. Elles permettent de représenter les transitions, les partitions et les états de manière intuitive tout en garantissant des performances optimales pour les opérations fréquentes, comme la recherche ou la mise à jour des données. Ce choix facilite également la lisibilité et la maintenance du code.

V. Implémentation de l'algorithme de minimisation

Dans cette implémentation, nous avons développé un programme en Java permettant de minimiser un automate fini déterministe (AFD). Le programme commence par lire les données d'un fichier texte décrivant l'AFD (nombre d'états, état initial, états finaux et transitions). Ces données sont ensuite organisées à l'aide

de structures Java telles que des Map et des Set, ce qui permet une gestion claire et efficace des états et des transitions.

L'algorithme de minimisation repose sur la séparation des états en groupes distincts (états finaux et non finaux), suivie d'un raffinage itératif pour identifier les états équivalents. Une fois les partitions stabilisées, un nouvel automate minimal est construit en regroupant les états équivalents et en redéfinissant les transitions.

Enfin, le programme affiche les états, transitions, état initial, et états finaux de l'automate minimal, offrant une vérification claire et lisible du résultat obtenu.

Description de notre automate AFD :

- **Nombre total d'états :** `5` → États : {0, 1, 2, 3, 4}.
- **État initial :** `1`.
- **États finaux :** `0` et `4`.
- **Transitions :** les transitions décrivent le comportement de l'AFD pour chaque symbole lu.

Voici notre fichier afn.txt contenant la description de notre AFD ou la représentation de l'AFD :

```
a 0 1, b 0 2 : Transitions de 0 vers 1 sur a et transition de 0 vers 2 sur b.

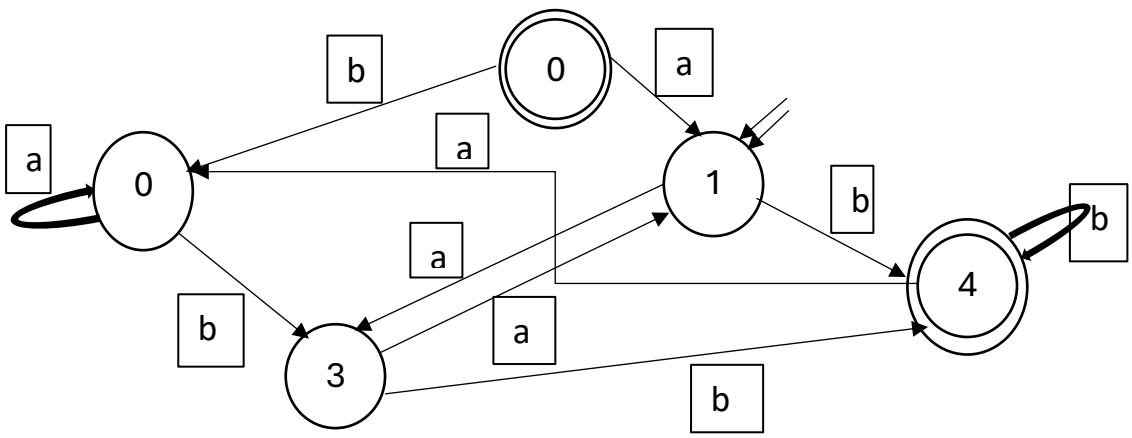
a 1 3, b 1 4: Transitions de 1 vers 3 sur a et transition de 1 vers 4 sur b.

a 2 2, b 2 3 : Boucle sur 2 pour et transition de 2 vers 3 sur b.

a 3 1, b 3 4 : Transitions de 3 vers 1 sur a et transition de 3 vers 4 sur b.

a 4 2, b 4 4 : Transition de 4 vers 1 sur a et boucle sur l'état 4 pour b.
```

Voici notre automate AFD :



Description de notre automate minimiser :

Étape 1 : Partition initiale

- États finaux : `{0, 4}`
- États non finaux : `{1, 2, 3}`

Étape 2 : Refinement 5

Analysons les transitions pour distinguer les états :

    - État 0 :
        - Avec a, va à 1 (non final).
        - Avec b, va à 2 (non final).
    - État 4 :
        - Avec a, va à 2 (non final).
        - Avec b, reste à 4 (final).
    - Résultat : {0} et {4} sont dans des groupes différents.
- Pour les états {1, 2, 3} :
    - État 1 :
        - Avec a, va à 3 (non final).
        - Avec b, va à 4 (final).
    - État 2 :
        - Avec a, reste à 2 (non final).
        - Avec b, va à 3 (non final).
    - État 3 :
        - Avec a, va à 1 (non final).
        - Avec b, va à 4 (final).
    - Résultat : {1, 3} et {2} sont dans des groupes différents.

Les Nouvelles partitions sont : {0},{4}, {1, 3}et {2}

Étape 3 : Création de l'AFD minimal

- **États minimaux :** S0 (pour {0}), S1 (pour {4}), S2 (pour {1, 3}), S3 (pour {2}).
- **État initial** : S2 (correspond à l'état 1).
- **États finaux :** S0 et S1.
- **Les Transitions minimales :**

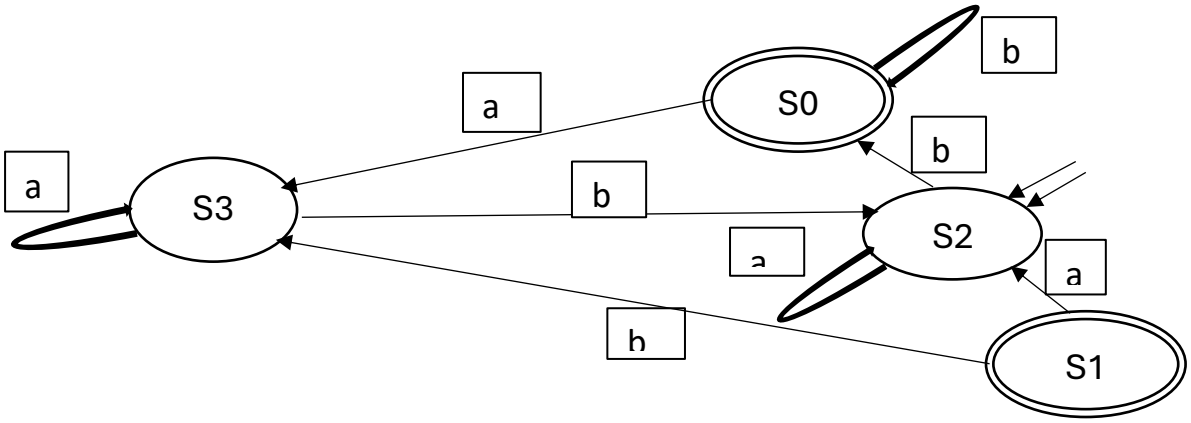
a S0 S3, b S0 S0: Transition de S0 vers S3 sur a et boucle sur S0 pour b

a S1 S2, b S1 S3: Transitions de S1 vers S3 sur b et transition de S1 vers S2 sur a.

a S2 S2, b S2 S0 : Boucle sur S2 pour a et transition de S2 vers S0 sur b.

a S3 S3, b S3 S2 : Boucle sur S3 pour a et transition de S3 vers S2 sur b.

Voici l'automate minimisé de l'AFD : La minimisation d'un AFD consiste à réduire le nombre d'états tout en préservant son comportement.



Pour exécuter le code : javac MinimiserAFD.java, pour compiler : java MinimiserAFD

Voici le résultat de notre automate :

```
PS C:\Users\ahmat\OneDrive\Documents\Tp_Grammaire&Langage_Djibrine_Moustapha&Khadidja_Ahmat>  & 'C:\Program Files\Jav
23\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\ahmat\AppData\Roaming\C
er\workspaceStorage\3d200bca64bc91065df7928c19ec62ea\redhat.java\jdt_ws\Tp_Grammaire&Langage_Djibrine_Moustapha&Khadi
mat_84e4e9cf\bin' 'MinimiserAFD'
États de l'AFD minimal : [S3, S0, S1, S2]
État initial : S2
États finaux : [S0, S1]
Transitions :
S3 a S3
S3 b S2
S0 a S3
S0 b S0
S1 a S2
S1 b S3
S2 a S2
S2 b S0
PS C:\Users\ahmat\OneDrive\Documents\Tp_Grammaire&Langage_Djibrine_Moustapha&Khadidja_Ahmat>
```

Sources de notre recherche sur les automates :

- Conversion de NFA à DFA - GeeksforGeeks
- Minimisation de DFA - GeeksforGeeks
- Tutoriel Java : Apprendre la programmation Java

Conclusion

En conclusion, ce rapport met en lumière le développement de deux algorithmes essentiels pour le traitement des automates finis : la conversion d'un AFN en AFD et la minimisation d'un AFD. Ces algorithmes, conçus en Java, permettent de manipuler efficacement les automates tout en garantissant la clarté des entrées (fichiers texte descriptifs) et des sorties (automates transformés).